



Point-Cloud & Multi-Tenant Server Build

A single Threadripper PRO box running Proxmox VE, serving four workloads at once. This guide walks the whole build — sequenced so your friend's Windows machine comes up first.

Proxmox VE 9.x

Threadripper PRO 9965WX · 24c/48t

192 GB ECC

RTX 5080 + 3080 passthrough

MikroTik RB5009 · VLANs

Windows-first

0 Where we are & how to read this

Proxmox VE is installed on the server, and the server lives at your friend's place with him. You're holding the MikroTik at your bench. So the job now is: pre-stage the network here, finish host prep, then stand up the Windows VM first, then everything else.

DONE SO FAR

Hardware assembled · Proxmox VE installed on the ZFS mirror · you can reach the Proxmox web UI. That's the end of *Phase 1–2* below.

The order of operations

Phase	What	Where	Status
0	Pre-stage MikroTik VLANs	Your bench	Do now (in parallel)
1	BIOS / firmware / IPMI lifeline	Friend's (server)	Verify settings
2	Proxmox host + Ansible prep (VFIO, ARC, NUT, Tailscale)	Friend's (server)	Installed → run prep
3	Ship router · wire VLANs · vmbr0 VLAN-aware	Friend's	After Phase 0
★ 4	Windows-main VM (the friend's machine)	Friend's	First VM up
5	Services · DayZ · Linux-compile VMs	Friend's	After Windows works
6	Resilience: NUT + Proxmox Backup Server	Friend's	Before "done"
7	Validation pass	Both	Sign-off

HOW TO READ EACH PHASE

Every phase has a **goal**, the **steps**, and a **“green light” check** you must pass before moving on. Don't skip a green light — especially the IPMI one in Phase 1. It's the only way back into a box that's three rooms (or a city) away.

• The system at a glance

CPU

AMD Threadripper PRO 9965WX — 24c / 48t, sTR5 / WRX90

Board

ASUS Pro WS WRX90E-SAGE SE — 8-ch ECC, 128 PCIe5 lanes, 10 GbE, AST2600 BMC + IPMI

RAM

192 GB = 8×24 GB DDR5-6000 ECC RDIMM

GPUs

RTX 5080 16 GB → Windows-main · RTX 3080 10 GB → Services

Storage

2×2 TB Samsung 9100 PRO (ZFS mirror) + 1×4 TB Gen4 M.2 (scratch)

Power / cooling

Corsair HX1500i · Noctua NH-U14S TR5-SP6 · Corsair 9000D · APC SMT1500C

Edge

MikroTik RB5009UPr+S+IN — firewall, inter-VLAN routing, 10G SFP+ trunk

Backup (separate box)

Recased workstation · 64 GB · 3×16 TB RAIDZ1 · RTX 5070 Ti · Proxmox Backup Server

What runs where

```
Proxmox VE host · 9965WX / 192 GB / 5080 + 3080
|
|-- Windows-main  5080 passthrough · Trion + point cloud + gaming · Sunshine host
|                  ~136-140 GB LOCKED · ballooning OFF · ~12-14 cores (float) [VLAN 30]
|
|-- Linux compile  NixOS · NO GPU · balloon 16->150 GB · 4-6 cores pinned      [VLAN 10]
|                  Penn's remote SSH kernel-build box
|
|-- Services       3080 passthrough (NVENC) · Linux · ~16 GB · 2 cores          [VLAN 20]
|                  Jellyfin + thin DBs
|
|-- DayZ           Windows · no GPU · ~8 GB · 2-3 cores · Deer Isle + Expansion [VLAN 20]
|
|-- Host           BMC/IPMI console (no GPU consumed) · ~2 cores · ZFS ARC ~8 GB [VLAN 10]
```

THE ONE RULE THAT EXPLAINS THE WHOLE RAM PLAN

GPU passthrough **pins** a VM's RAM for DMA, so a passthrough VM **cannot balloon**. Windows-main's ~138 GB only returns to the pool when that VM is **powered off** — not when the friend closes Moonlight, not when Windows is idle or asleep. That's why hugepages are on-demand (not boot-reserved) and why the Linux compile VM only grows past its ~16 GB floor when you deliberately shut Windows down ("Big-Linux mode").

Resource budget (all four VMs running)

VM	RAM	Cores	GPU	Balloon
Host + ZFS ARC	~10 GB	~2	—	ARC capped ~8 GB
Windows-main	~136–140 GB	~12–14 (float)	RTX 5080	OFF (locked)
Services	~16 GB	~2	RTX 3080	OFF (locked)
DayZ	~8 GB	2–3	—	optional
Linux compile	16 → 150 GB	4–6 pinned	—	ON

Committed ≈ 188 GB, ~4 GB slack. Windows + Linux never both want their max — that's by design.

0 Pre-stage the MikroTik at your bench

Yes — you can absolutely build the VLANs now at your place, then ship the router to your friend. RouterOS config lives on the device, so a router you configure here arrives ready to drop in. This is the smart move: do the fiddly, lock-out-prone part on a bench where a mistake costs nothing.

WHAT "PRE-STAGING" MEANS HERE

You power the RB5009 on your desk, push the full VLAN + firewall config, verify it boots clean, then box it up. At the friend's you just plug WAN + the trunk + access ports into the right jacks and it works. **Nothing in the config depends on being physically at the friend's** except the WAN DHCP lease, which it picks up on arrival.

The VLAN map you're staging

VLAN	Name	Subnet	Members
10	mgmt	10.10.10.0/24	Router, PVE host mgmt, BMC/IPMI, PBS, Linux-compile VM
20	servers	10.10.20.0/24	Services VM (Jellyfin/DBs), DayZ VM, backup box
30	gaming	10.10.30.0/24	Windows-main VM + friend's mini PC
40	trusted	10.10.40.0/24	Penn's laptop, workstations, spare ports

WHY THE MINI PC AND WINDOWS-MAIN SHARE VLAN 30

Same broadcast domain = Moonlight auto-discovery (mDNS) works, and the latency-critical stream is **L2 hardware-switched** — it never touches the router CPU. That's how you get sub-millisecond in-room streaming. Keep them together.

Bench procedure

1. **Update firmware first.** In Winbox: `/system package update` then `/system routerboard upgrade`, reboot between. Set a strong admin password; disable telnet/ftp/www/api-ssl.
2. **Connect via Winbox over MAC** (not IP) — it survives the IP changes you're about to cause, so you can't lock yourself out mid-config.
3. **Paste the full config** from `docs/network-topology.md` — bridge, ports, VLAN membership, gateways, DHCP, NAT, firewall — **but leave the very last line off.**
4. **Verify you still have access**, then enable VLAN filtering *last*:

```
# ONLY after everything else is in and you've confirmed access:
/interface bridge set bridge vlan-filtering=yes
```

CLASSIC LOCK-OUT

Turning on `vlan-filtering=yes` with a wrong tagged/untagged map cuts your own connection. On a bench that's a 30-second power-cycle; at the friend's with the router in a closet it's a bad evening. **This is the entire reason to stage at home.**

Green light before you box it up

- `/interface bridge vlan print` shows the tagged/untagged map you expect.
- `/ip address print` shows all four gateways (...10.1, .20.1, .30.1, .40.1).
- A laptop on `ether2` gets a `10.10.30.x` lease; on `ether5` gets `10.10.40.x`.
- That laptop can reach the internet through the router.

Label the ports with tape (WAN / trunk / mini-PC / IPMI / backup) so the friend-side plug-in is foolproof, then ship it.

1 BIOS, firmware & the IPMI lifeline

Since Proxmox is already installed, most of this is a verify pass — but confirm these or passthrough will fail later in ways that are hard to debug remotely.

BIOS settings to confirm

Setting	Value	Why
IOMMU / AMD-Vi	Enabled	Required for any GPU passthrough
SR-IOV / ACS	Enabled (if present)	Cleaner IOMMU group isolation
Above-4G Decoding	Enabled	Large-BAR GPUs (5080 16 GB) need it
Resizable BAR	Enabled	Blackwell 5080 PCIe enumeration
Fan curves	24/7 sane profile	Box runs unattended at the friend's

★ THE IPMI LIFELINE — DO NOT SKIP

The WRX90E-SAGE SE has an AST2600 BMC with a dedicated IPMI port. Get it on **VLAN 10 (mgmt)**, set a password, and **confirm the remote KVM (HTML5 console) actually works from another machine** before you rely on it. When passthrough binds both GPUs to vfio, the host has *no local video* — IPMI is your only console. Verify it's populated on the *SE* variant specifically; that's an open item in the spec.

Green light: you can open the IPMI HTML5 console from a second machine, see the Proxmox login, and "mount ISO" works. That's your remote hands.

2 Proxmox host prep (VFIO, ARC, NUT, Tailscale)

Proxmox is installed; now configure the host-level things OpenTofu can't manage. This is exactly what the `ansible/` role in this repo does — one playbook does VFIO binding, ARC cap, NUT, and Tailscale.

1 Discovery — collect the hardware-specific values (run on the host)

```
lspci -nn | grep -iE 'nvidia|audio'  # -> the PCI IDs: GPU + audio fn of EACH card
ip -br link                          # -> the NIC name (10G)
ls -l /dev/disk/by-id/               # -> stable disk paths if needed
```

Each NVIDIA card shows **two** functions — the GPU and its HDMI-audio device. You pass *both*. So you'll list four IDs total (5080 GPU + 5080 audio, 3080 GPU + 3080 audio).

2 Fill the repo vars

- `ansible/group_vars/all.yml` → `vfio_pci_ids` (the four device-IDs).
- `ansible/inventory.ini` → the host's mgmt IP.

3 Run the play (pass secrets at runtime — never commit them)

```
cd ansible
ansible-playbook site.yml \
  -e "tailscale_authkey=tskey-XXXX nut_monitor_password=SOMETHING"
```

This sets the kernel cmdline via **proxmox-boot-tool** (ZFS root uses systemd-boot, not grub — important detail), binds vfio-pci, blacklists the NVIDIA drivers on the host, caps `zfs_arc_max` ~8 GB, installs NUT against the APC, and brings up Tailscale.

4 Reboot, then verify

```
./scripts/verify-passthrough.sh
```

DO NOT PROCEED UNTIL GREEN

Every GPU *and* audio function must show `Kernel driver in use: vfio-pci`, each card in a **clean, isolated IOMMU group**, ARC capped, NUT talking to the UPS, Tailscale up. If a GPU is sharing an IOMMU group with something else, fix that (ACS / slot choice) *before* building VMs — a dirty group means passthrough will be flaky in ways that look like random VM crashes.

Green light: both GPUs on `vfio-pci` in clean groups · ARC capped · NUT + Tailscale healthy.

3 Wire the network at the friend's

The router arrives pre-staged from Phase 0. Now physically wire it and make the Proxmox bridge VLAN-aware so each VM lands in the right segment.

Plug-in map (match your tape labels)

Port	Goes to	VLAN
ether1	ISP / WAN (DHCP)	—
sfp-sfpplus1 (10G)	Proxmox host 10G NIC (trunk)	10,20,30,40 tagged
ether2	Friend's mini PC (Moonlight)	30 untagged
ether3	Backup / ZFS box	20 untagged
ether4	Host BMC / IPMI	10 untagged
ether5–8	Spare / clients	40 untagged

Make vbr0 VLAN-aware (Proxmox)

In the Proxmox UI: *Datacenter* → *host* → *System* → *Network* → edit `vbr0` (the bridge on the 10G NIC) → tick **VLAN aware**. Then each VM's vNIC gets a **VLAN Tag** set to its segment (Windows = 30, Services/DayZ = 20, Linux-compile = 10). The host's own mgmt IP sits on VLAN 10.

PUT THE HOST MGMT + IPMI ON TAILSCALE

Install Tailscale on the host (the Ansible role did this) and confirm you can reach both the Proxmox UI and the IPMI console over Tailscale from outside. That's your out-of-band recovery path for a box you don't physically sit next to.

Green light: mini PC on ether2 pulls a `10.10.30.x` lease · host reachable on VLAN 10 · Tailscale reaches host + IPMI · internet works from each VLAN.

4 ★ The Windows-main VM (the friend's machine)

This is the one your friend actually uses — Trion, point-cloud work, and gaming, all streamed to his in-room mini PC. It's the first VM you build and the most detailed section here. Take your time; get this rock-solid before touching the others.

★ WHICH WINDOWS DOES HE NEED? — WINDOWS 11 PRO (NOT HOME)

This isn't a preference, it's a hard requirement: **Windows 11 Home caps at 128 GB of RAM. This VM gets ~138 GB.** Home would simply refuse to address the extra memory — the whole point of the build (holding a 120 GB+ point cloud in RAM) breaks. Pro supports up to 2 TB, plus it gives you RDP host and cleaner headless behavior. Get a **retail Windows 11 Pro** license. (Pro for Workstations isn't needed — that's for ReFS / 4-socket / >2 TB; plain Pro covers you.)

What you'll need on hand

Windows 11 Pro ISO

Official Microsoft ISO, uploaded to Proxmox storage

VirtIO drivers ISO

Fedora `virtio-win.iso` — Windows can't see a virtio disk/NIC during install without it

Dummy HDMI plug

Plugged into the 5080 — gives Windows a "real" display for Sunshine to capture

Win 11 Pro license key

Retail key — see the RAM note above

1 Create the VM shell (Windows 11 needs UEFI + TPM)

In Proxmox *Create VM*, set these — Windows 11 will refuse to install without the TPM and UEFI pieces:

Setting	Value	Note
Machine	q35	Required for clean PCIe passthrough
BIOS	OVMF (UEFI)	+ add an EFI disk on the ZFS mirror
TPM	Add TPM v2.0 state	Win 11 hard requirement
CPU type	host	Exposes full TR-PRO feature set
Cores	~12-14	Keep SMT siblings together, contiguous in a CCD
Memory	~138 GB · ballooning OFF	Passthrough needs locked RAM
SCSI controller	VirtIO SCSI single	Fast disk; needs the virtio ISO at install
Disk	VirtIO, on the ZFS mirror	OS lives on the backed-up mirror
Network	VirtIO, VLAN tag 30	Same segment as the mini PC

2 Install Windows

- Attach **two** CD drives: the Windows 11 Pro ISO and the virtio-win ISO.
- When the installer shows no disks, click *Load driver* → browse the virtio ISO → load the **vioscsi** (storage) driver. The disk appears.
- Also load the **NetKVM** (network) driver so networking works post-install.
- Install Windows 11 Pro, enter the key.

3 Add the 5080 passthrough

- VM *Hardware* → *Add* → *PCI Device*: the **5080 GPU**, tick *All Functions* (grabs the HDMI-audio fn too), *PCI-Express*, and *Primary GPU*.
- Boot. Install the current **NVIDIA driver** inside Windows.

BLACKWELL + CONSUMER PASSTHROUGH

Modern NVIDIA drivers officially support consumer-card passthrough — no "Code 43" hack needed. Keep the KVM `hidden` flag handy only if a driver ever misbehaves. The 5080 is new silicon; the recent Proxmox kernel (Phase 1) is what gives it clean PCIe enumeration. On a VM reboot it usually FLR-resets fine; worst case a host reboot clears a stuck GPU, recoverable over IPMI.

4 Make it a capturable, always-ready desktop

- **Dummy HDMI plug** into the 5080 (preferred over a virtual display driver) so Windows always has a display to render/capture.
- Enable **auto-login** so a power-cycle boots straight to a ready desktop — the friend never sees a lock screen.
- Set display resolution to the target (4K60 or 1080p120).

5 Install Trion (point-cloud)

- Install Trion + the point-cloud toolchain.
- **If Trion uses a USB license dongle**, pass the physical USB device through to this VM (*Hardware* → *Add* → *USB Device*).

★ HIGHEST-RISK OPEN ITEM — VERIFY EARLY

Confirm **Trion actually runs under virtualization** and that its dongle (if any) passes through cleanly. The spec flags this as the #1 risk. Test it the moment Windows + GPU are up — before you've built everything else on the assumption that it works.

6 Sunshine + Moonlight (the in-room stream)

- Install **Sunshine** on Windows-main — the 5080's NVENC makes encode essentially free.
- On the friend's **mini PC** (wired to ether2, VLAN 30): install **Moonlight**, set it to auto-launch fullscreen on boot.
- Because both are on VLAN 30, Moonlight auto-discovers the host; pair them (PIN).

- Target **4K60 or 1080p120, sub-10 ms** added latency on the wired LAN.

★ GREEN LIGHT FOR PHASE 4

Friend sits at the mini PC, it boots straight into a fullscreen Moonlight session, the Windows desktop appears at target res/latency, **Trion launches**, and a real point-cloud job runs. That's the whole reason for the build — get a clean pass here before moving on.

5 The other three VMs

With Windows solid, build these via OpenTofu ([opentofu/](#) , bpg/proxmox) — a manual first pass in the UI is fine too. Each is lighter than the Windows VM.

Services VM (Linux) — Jellyfin + thin DBs

- Linux, ~16 GB (locked, since the 3080 is passed here), ~2 cores.
- Pass the **RTX 3080 + its audio fn** through; Jellyfin uses NVENC for transcoding.
- vNIC on **VLAN 20**. Expose *only* via Tailscale — never WAN.

DayZ VM (Windows) — Deer Isle + Expansion

- Small Windows VM, no GPU, ~8 GB, 2–3 cores (DayZ is largely single-thread-bound — favors clock). vNIC on **VLAN 20**.
- SteamCMD dedicated server + CF (Community Framework) + DayZ-Expansion-Core / -Vehicles / -Market + **Deer Isle**, Expansion mission adapted to Deer Isle.
- Required features: Deer Isle map · **fly-in safezone**s (Expansion zone-based, position+radius, traders inside — *not* teleport) · **helicopters** (Expansion-Vehicles).
- Outside players reach it via Tailscale, or the tightly-scoped WAN dst-nat (UDP 2302–2306 + 27016) already in the router config — never raw-exposed beyond those ports.

Linux compile VM (NixOS) — your remote kernel box

- NixOS (decided), no GPU, **ballooning ON** (floor 16 / max 150 GB), 4–6 cores pinned. vNIC on **VLAN 10**.
- Pinned RFL toolchain via a flake (exact rustc/bindgen/LLVM) — reproducible, atomic rollback so a remote box is very hard to brick.
- SSH/mosh in over Tailscale. The 50 Mbps uplink is a non-issue for compiles.

BIG-LINUX MODE (HOW YOU RECLAIM RAM FOR HUGE BUILDS)

Default is co-resident: Windows up 24/7, Linux at its ~16 GB floor — fine for most builds since -j parallelism is CPU-bound. For a giant tmpfs build: over SSH/IPMI confirm no active Sunshine session → `qm shutdown <winvm>` → run the job (Linux balloons up automatically) → `qm start <winvm>` to restore the friend's machine before he needs it. The friend never manages any of this.

6 Resilience & backups

- **NUT against the APC SMT1500C** (Ansible configured it) — and actually **test a simulated power-loss shutdown**. Mandatory for an unattended remote box. The 1000 W UPS is graceful-shutdown headroom, not ride-through.
- **Proxmox Backup Server** on the 64 GB / RAIDZ1 box — kept on the **local LAN** so bulk backups run at 10G speed (a full 120 GB over the 50 Mbps uplink is ~5.5 h — impractical). Run a first VM backup.
- **Offsite** via physical cold-drive rotation: `zfs send | zstd` to a bare drive in a USB dock, stored cold. Only small incrementals cross the WAN. That's your 3-2-1.

STORAGE DISCIPLINE

The friend's real project files and point clouds live on the **backed-up ZFS mirror**, never on the 4 TB scratch drive (no redundancy, transient only). Make this a habit from day one.

7 Validation & sign-off

Who	Check
Friend	Full Moonlight session at target res/latency · Trion launches · a real point-cloud job runs
Penn	SSH compile from outside via Tailscale · Jellyfin stream within the uplink cap · trigger Big-Linux mode once (Windows off → Linux balloons → <code>qm start</code> Windows)
Both	Simulated power-loss → graceful shutdown · first PBS backup restores cleanly

! Open items to verify (carry these in your head)

#	Item	Risk
1	Trion runs under virtualization + dongle passes through	HIGH — verify in Phase 4 before relying on the VM
2	IPMI/BMC populated on the SE board variant	High — it's your only console
3	RAIDZ1 backup box is LAN-local	Med — else bulk backup over 50 Mbps isn't viable
4	IOMMU groups isolate 5080 & 3080 cleanly	Med — dirty groups = flaky passthrough
5	CXL enablement on 9965WX + WRX90E firmware	Low — keep a PCIe5 x16 slot free for later
6	Both Gen5 NVMe have adequate heatsinking for 24/7	Low — they run hot

⌘ Command cheat sheet

On the Proxmox host

```
# discover passthrough IDs / NIC / disks
lspci -nn | grep -iE 'nvidia|audio'
ip -br link
# host prep (this repo)
cd ansible && ansible-playbook site.yml -e "tailscale_authkey=... nut_monitor_password=..."
./scripts/verify-passthrough.sh # must be all-green before VMs
# Big-Linux mode
qm shutdown <winvm> # Linux balloons up
qm start <winvm> # restore friend's machine
```

On the MikroTik (bench)

```
/interface bridge vlan print # tagged/untagged map
/ip address print # four gateways
/ip dhcp-server lease print # clients in the right subnet
/interface bridge set bridge vlan-filtering=yes # LAST, after verifying access
```

TL;DR FOR YOUR FRIEND

His machine is a Windows 11 **Pro** VM with the 5080 passed straight through, ~138 GB of RAM, streamed to his mini PC over the in-room wired LAN with Sunshine → Moonlight. He turns the mini PC on, it boots into his desktop. Everything else on the box is invisible to him.